
18 B-Trees

B-trees are balanced search trees designed to work well on disk drives or other direct-access secondary storage devices. B-trees are similar to red-black trees (Chapter 13), but they are better at minimizing the number of operations that access disks. (We often say just “disk” instead of “disk drive.”) Many database systems use B-trees, or variants of B-trees, to store information.

B-trees differ from red-black trees in that B-tree nodes may have many children, from a few to thousands. That is, the “branching factor” of a B-tree can be quite large, although it usually depends on characteristics of the disk drive used. B-trees are similar to red-black trees in that every n -node B-tree has height $O(\lg n)$, so that B-trees can implement many dynamic-set operations in $O(\lg n)$ time. But a B-tree has a larger branching factor than a red-black tree, so the base of the logarithm that expresses its height is larger, and hence its height can be considerably lower.

B-trees generalize binary search trees in a natural manner. Figure 18.1 shows a simple B-tree. If an internal B-tree node x contains $x.n$ keys, then x has $x.n + 1$ children. The keys in node x serve as dividing points separating the range of keys handled by x into $x.n + 1$ subranges, each handled by one child of x . A search for a key in a B-tree makes an $(x.n + 1)$ -way decision based on comparisons with the $x.n$ keys stored at node x . An internal node contains pointers to its children, but a leaf node does not.

Section 18.1 gives a precise definition of B-trees and proves that the height of a B-tree grows only logarithmically with the number of nodes

it contains. Section 18.2 describes how to search for a key and insert a key into a B-tree, and Section 18.3 discusses deletion. Before proceeding, however, we need to ask why we evaluate data structures designed to work on a disk drive differently from data structures designed to work in main random-access memory.

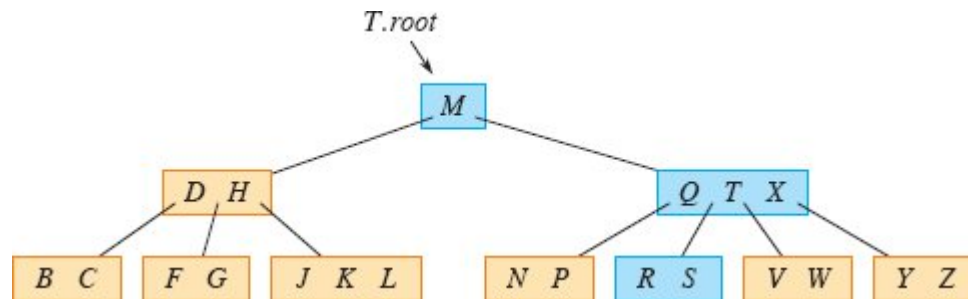


Figure 18.1 A B-tree whose keys are the consonants of English. An internal node x containing $x.n$ keys has $x.n + 1$ children. All leaves are at the same depth in the tree. The blue nodes are examined in a search for the letter R .

Data structures on secondary storage

Computer systems take advantage of various technologies that provide memory capacity. The *main memory* of a computer system normally consists of silicon memory chips. This technology is typically more than an order of magnitude more expensive per bit stored than magnetic storage technology, such as tapes or disk drives. Most computer systems also have *secondary storage* based on solid-state drives (SSDs) or magnetic disk drives. The amount of such secondary storage often exceeds the amount of primary memory by one to two orders of magnitude. SSDs have faster access times than magnetic disk drives, which are mechanical devices. In recent years, SSD capacities have increased while their prices have decreased. Magnetic disk drives typically have much higher capacities than SSDs, and they remain a more cost-effective means for storing massive amounts of information. Disk drives that store several terabytes¹ can be found for under \$100.

Figure 18.2 shows a typical disk drive. The drive consists of one or more *platters*, which rotate at a constant speed around a common *spindle*. A magnetizable material covers the surface of each platter. The

drive reads and writes each platter by a *head* at the end of an *arm*. The arms can move their heads toward or away from the spindle. The surface that passes underneath a given head when it is stationary is called a *track*.

Although disk drives are cheaper and have higher capacity than main memory, they are much, much slower because they have moving mechanical parts. The mechanical motion has two components: platter rotation and arm movement. As of this writing, commodity disk drives rotate at speeds of 5400–15,000 revolutions per minute (RPM). Typical speeds are 15,000 RPM in server-grade drives, 7200 RPM in drives for desktops, and 5400 RPM in drives for laptops. Although 7200 RPM may seem fast, one rotation takes 8.33 milliseconds, which is over 5 orders of magnitude longer than the 50 nanosecond access times (more or less) commonly found for main memory. In other words, if a computer waits a full rotation for a particular item to come under the read/write head, it could access main memory more than 100,000 times during that span. The average wait is only half a rotation, but still, the difference in access times for main memory compared with disk drives is enormous. Moving the arms also takes some time. As of this writing, average access times for commodity disk drives are around 4 milliseconds.

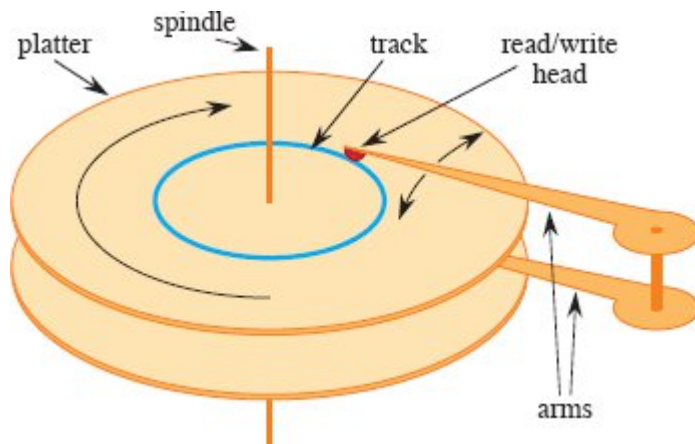


Figure 18.2 A typical magnetic disk drive. It consists of one or more platters covered with a magnetizable material (two platters are shown here) that rotate around a spindle. Each platter is read and written with a head, shown in red, at the end of an arm. Arms rotate around a common pivot axis. A track, drawn in blue, is the surface that passes beneath the read/write head when the head is stationary.

In order to amortize the time spent waiting for mechanical movements, also known as *latency*, disk drives access not just one item but several at a time. Information is divided into a number of equal-sized *blocks* of bits that appear consecutively within tracks, and each disk read or write is of one or more entire blocks.² Typical disk drives have block sizes running from 512 to 4096 bytes. Once the read/write head is positioned correctly and the platter has rotated to the beginning of the desired block, reading or writing a magnetic disk drive is entirely electronic (aside from the rotation of the platter), and the disk drive can quickly read or write large amounts of data.

Often, accessing a block of information and reading it from a disk drive takes longer than processing all the information read. For this reason, in this chapter we'll look separately at the two principal components of the running time:

- the number of disk accesses, and
- the CPU (computing) time.

We measure the number of disk accesses in terms of the number of blocks of information that need to be read from or written to the disk

drive. Although disk-access time is not constant—it depends on the distance between the current track and the desired track and also on the initial rotational position of the platters—the number of blocks read or written provides a good first-order approximation of the total time spent accessing the disk drive.

In a typical B-tree application, the amount of data handled is so large that all the data do not fit into main memory at once. The B-tree algorithms copy selected blocks from disk into main memory as needed and write back onto disk the blocks that have changed. B-tree algorithms keep only a constant number of blocks in main memory at any time, and thus the size of main memory does not limit the size of B-trees that can be handled.

B-tree procedures need to be able to read information from disk into main memory and write information from main memory to disk. Consider some object x . If x is currently in the computer's main memory, then the code can refer to the attributes of x as usual: $x.key$, for example. If x resides on disk, however, then the procedure must perform the operation $\text{DISK-READ}(x)$ to read the block containing object x into main memory before it can refer to x 's attributes. (Assume that if x is already in main memory, then $\text{DISK-READ}(x)$ requires no disk accesses: it is a “no-op.”) Similarly, procedures call $\text{DISK-WRITE}(x)$ to save any changes that have been made to the attributes of object x by writing to disk the block containing x . Thus, the typical pattern for working with an object is as follows:

x = a pointer to some object
 $\text{DISK-READ}(x)$
operations that access and/or modify the attributes of x
 $\text{DISK-WRITE}(x)$ // omitted if no attributes of x were changed
other operations that access but do not modify attributes of x

The system can keep only a limited number of blocks in main memory at any one time. Our B-tree algorithms assume that the system automatically flushes from main memory blocks that are no longer in use.